

INT332 – DevOps

Unit 3 – Week 7

Microservices Fundamentals

Subject Information

Subject: INT332 DevOps
Unit: 3
Week: 7
Topic: Microservices Fundamentals

Learning Outcomes

After studying this topic, students will be able to:

- Define microservices architecture.
- Explain why microservices are needed.
- Describe the characteristics of microservices.
- Identify benefits and challenges of adopting microservices.

1. Introduction to Microservices

Microservices is a software architecture style in which an application is built as a collection of small, independent services. Each service focuses on a single business capability and can be developed, tested, deployed, and scaled independently. The services communicate using APIs such as REST, gRPC, or messaging systems. Unlike traditional applications where all components are packaged together, microservices break the application into manageable pieces. This approach improves flexibility and supports modern cloud-native development practices.

2. Why Microservices Are Needed

As software systems grow, monolithic applications become difficult to maintain. A single change may require rebuilding and redeploying the entire application. Teams working on different modules can interfere with one another, slowing development. Microservices were introduced to solve these problems by dividing applications into smaller independent services. Each service can evolve separately, allowing faster development and deployment.

3. Key Characteristics of Microservices

1. Single Responsibility: Each service performs one business function.
2. Independent Deployment: Services can be deployed without affecting others.
3. Loose Coupling: Services have minimal dependencies.
4. Decentralized Data Management: Each service may manage its own database.
5. Technology Diversity: Different services can use different programming languages and databases.
6. Fault Isolation: Failure of one service does not necessarily stop the entire system.

4. Working of a Microservices-Based Application

Consider an e-commerce application. Instead of having one large application, it may contain:

- User Service
 - Product Service
 - Order Service
 - Payment Service
 - Notification Service
- Each service performs its own task and communicates with other services through APIs.

5. Advantages of Microservices

Scalability: Individual services can be scaled based on demand.

Faster Development: Multiple teams can work on different services simultaneously.

Better Reliability: Failure in one service has limited impact on others.

Easier Maintenance: Smaller codebases are easier to understand and modify.

Technology Flexibility: Teams can choose the most suitable technology stack for each service.

6. Challenges of Microservices

Increased Complexity: Managing many services is more difficult than managing one application.

Network Communication: Services depend on network calls, which may introduce latency.

Monitoring and Logging: Tracking issues across multiple services requires specialized tools.

Security Management: More APIs create additional security considerations.

Data Consistency: Maintaining consistency across multiple databases can be challenging.

7. Real-World Use Cases

Netflix uses microservices to stream content globally.

Amazon uses microservices to manage products, payments, orders, and customer accounts.

Uber uses microservices for ride management, payments, notifications, and mapping services.

Exam-Oriented Points

- Microservices divide applications into small independent services.
- Each service focuses on a single business capability.
- Services communicate using APIs.
- Microservices improve scalability, agility, and maintainability.
- They are widely used in cloud-native applications.

Viva Questions

1. What is a microservice?
2. Why are microservices preferred over large applications?
3. What is loose coupling?
4. What are the advantages of microservices?
5. Name some companies that use microservices.

Summary

Microservices architecture organizes applications into small, independent services that work together to deliver business functionality. This approach improves scalability, flexibility, deployment speed, and maintainability. Although microservices introduce operational complexity, they are one of the most widely adopted architectures in modern software development and DevOps.